

Problem Set 2

due by 11:59 p.m. on Tuesday, September 23, 2025

Preliminaries

In your work on this assignment, make sure to abide by the [collaboration policies](#) of the course.

If you have questions while working on this assignment, please come to office hours, post them on Piazza.

Make sure to follow the instructions outlined at the end of Part I and Part II when submitting your assignment.

Part I

45 points total

This part of the assignment consists of a series of short-answer questions. You will be able to check your answers to at least some of these questions by executing the code, but we strongly encourage you to answer them on your own and to convince yourself of the correctness of your answers before you try to test them. These questions are similar to ones that could appear on the midterms and final exam, so it's important to be able to answer them without the use of a computer.

Creating the necessary folder

1. If you haven't already created a folder named cs112 for your work in this course, follow [these instructions](#) to do so.
2. Then create a subfolder called ps2 within your cs112 folder, and put all of the files for this assignment in that folder.

Creating the necessary file

Problems in Part I will be completed in a single PDF file. To create it, you should do the following:

1. Access the template that we have created by clicking on [this link](#) and signing into your Google account as needed.
2. When asked, click on the *Make a copy* button, which will save a copy of the template file to your Google Drive.
3. Select *File->Rename*, and change the name of the file to ps2_partI.
4. Add your work for the problems from Part I to this file.
5. Once you have completed all of these problems, choose *File->Download->PDF document*, and save the PDF file in your ps2 folder. The resulting PDF file (ps2_partI.pdf) is the one that you will submit. See the submission guidelines at the end of Part I.

Problem 1: Variable scope

9 points total; individual-only

See the section above for guidelines that you should follow to create the file containing your answers to Part I.

Consider the following program, which includes a number of incomplete println statements:

```
public class ScopeThisOut {
```

```

public static void doSomething(int a) {
    int b = 5;
    if (a > b) {
        System.out.println(______);    // first println
        int c = 2;
    }

    for (int i = 1; i <= 5; i++) {
        System.out.println(______);    // second println
        for (int j = 0; j < 3; j++) {
            int d = 2;
            System.out.println(______);    // third println
        }
        int e = 4;
    }

    System.out.println(______);    // fourth println
}

public static void main(String[] args) {
    int f = 5;

    System.out.println(______);    // fifth println

    int g = 2;
    doSomething(f);

    System.out.println(______);    // sixth println
}
}

```

The program includes a number of `int` variables: `a`, `b`, `c`, `d`, `e`, `f`, `g`, `i`, and `j`. Given the rules that we have learned about variable scope:

1. Which of these variables could be printed by the first `println` statement?
2. Which of them could be printed by the second `println` statement?
3. Which of them could be printed by the third `println` statement?
4. Which of them could be printed by the fourth `println` statement?
5. Which of them could be printed by the fifth `println` statement?
6. Which of them could be printed by the sixth `println` statement?

Note that we are only focusing on the variables of type `int`, so you can ignore the `args` parameter of the `main` method.

Problem 2: String objects and their methods

Working with strings: Python vs. Java

Python has built-in operators and methods for common string operations like indexing and slicing. In addition, string objects — like all objects — have methods inside them. These methods allow you to obtain information about a string and to create new strings that are based on the original one.

Java lacks many of Python's built-in operators and functions for strings. Instead, most operations involving strings – including indexing and slicing – are done using methods that are inside the string object.

The table below compares the ways in which some common string operations would be accomplished in Python and Java:

operation	in Python	in Java
assigning to a variable (and, in Java, declaring the variable)	<code>s1 = 'Boston'</code> <code>s2 = "University"</code>	<code>String s1 = "Boston";</code> <code>String s2 = "University";</code>
concatenation	<code>s1 + s2</code>	<code>s1 + s2</code>
finding the number of characters	<code>len(s1)</code>	<code>s1.length()</code>
indexing (note: Java does <i>not</i> have negative indices)	<code>s1[0]</code> <code>s2[-1]</code>	<code>s1.charAt(0)</code> <code>s2.charAt(s2.length()-1)</code> (see the notes below the table for a key fact about this method)
slicing	<code>s1[1:4]</code> <code>s2[3:]</code> <code>s1[:5]</code>	<code>s1.substring(1, 4)</code> <code>s2.substring(3)</code> <code>s1.substring(0, 5)</code>
converting all letters to uppercase	<code>s1.upper()</code>	<code>s1.toUpperCase()</code>
converting all letters to lowercase	<code>s2.lower()</code>	<code>s2.toLowerCase()</code>
determining if a substring is in a string	<code>s2 in s1</code> <code>'sit' in s2</code>	<code>s1.contains(s2)</code> <code>s2.contains("sit")</code>
finding the index of the first occurrence of a character or substring	<code>s1.find("o")</code> <code>s2.find('ver')</code>	<code>s1.indexOf('o')</code> <code>s2.indexOf("ver")</code>
testing if two strings are	<code>s1 == s2</code>	<code>s1.equals(s2)</code>

equivalent	s1 == 'Boston'	s1.equals("Boston")
------------	----------------	---------------------

Notes:

- Recall that Java has a separate type called char for values that consist of a single character. Literals of this type are surrounded by single quotes (e.g., the 'o' in the call s1.indexOf('o')), whereas string literals must be surrounded by double quotes. (In Python, there is no separate char type, and string literals can be surrounded by either single or double quotes.)
- The charAt() method that we use for indexing in Java returns a value of type char. For example, the call s1.charAt(0) in the table above would return the char value 'B', since 'B' is the first character in the string "Boston".

Given the information above, here are the tasks you should perform:

0. (0 points) To ensure that you understand how the above string operations work in Java, we strongly encourage you to write a simple program to experiment with them.

For example, you could put the following lines in the main method of a test program:

```
String s1 = "Boston";
System.out.println( s1.substring(1, 4) );
```

Doing so should display the result of the expression being printed:

```
ost
```

1. (6 points) Assume that the following statements have already been executed:

```
String str1 = "Let's learn Java!";
String str2 = "Strings or Arrays?";
```

For each of the following values, construct an expression involving operations on str1 and/or str2 that evaluates to that value. **For full credit, you should not use literal values (e.g., "T" or 'e') unless it is absolutely necessary to do so.**

- a. Construct an expression involving str1 and/or str2 that produces the following value:

```
"Strings or Java!"
```

We have given you the answer to this part of the problem in ps2_partI.

Important: Note that the provided answer is *a single expression* that produces the necessary value. The answer does not use any assignment statements or print statements. Your answers to each of the remaining parts of this question should also consist of just a single expression.

- b. Construct an expression involving `str1` and/or `str2` that produces the following value:

"Stays"

- c. Construct an expression involving `str1` and/or `str2` that produces the following value:

"learn ARRAYS"

Hint: One or more parts of your expression will need to chain two method calls together – i.e., to do something that looks like this:

```
str2.toLowerCase().substring(9)
```

- d. Construct an expression involving `str1` and/or `str2` that produces the following value:

'v'

Note that the single quotes mean that you must produce a `char`, *not* a `String`. See the notes under the table above that discuss the `char` type.

- e. Construct an expression involving `str1` and/or `str2` that produces the following value:

"no"

- f. Construct an expression involving `str1` and/or `str2` that produces the following value:

17

Hint: Use the `indexOf` method to find a character in `str1` or `str2`.

- g. Construct an expression involving `str1` and/or `str2` that produces the following value:

"learn Strings or Strongs?"

Use the first replace method in the String class; consult the [API](#) of the String class to figure out how to use it.

Problem 3: Understanding code that uses an array

8 points total; *individual-only*

1. (4 points) Consider the following static method, which operates on an array of integers:

```
public static void mystery(int[] arr) {
    for (int i = 0; i < arr.length - 2; i++) {
        int val1 = arr[i + 1];
        int val2 = arr[i + 2];
        arr[i] = val2 - val1;

        // What values do the variables have here,
        // for each value of the loop variable `i`?
    }
}
```

Consider the following code fragment, which calls this method:

```
int[] values = {1, 4, 9, 16, 25, 36, 49, 64};
mystery(values);
```

In section 3-1 of ps2_partI, we've provided a table that you should complete to illustrate the execution of the loop during the above method call.

We have started the table for you. The first row shows what the array looks like before the start of the for loop. For each value of the loop variable *i*, include a row in the table that shows what the array looks like **at the end** of the iteration of the loop for that value of *i*. You may not need all of the rows of the table.

(Note: You should write the value of *values* as an array literal that shows the current contents of the array, even though the variable itself actually holds a reference to the array.)

2. (4 points) After running the code fragment from part 1, we then execute the following statement:

```
System.out.println(Arrays.toString(arr));
```

Do we see the original array, or do we see the changes made by the call to the `mystery()` method? Explain briefly *why* your answer makes sense.

(Note: If you want to check your answer, you'll need to import the `java.util` package at the top of your test class so that you can use the `Arrays.toString()` method.)

Problem 4: Arrays and memory diagrams

12 points; **individual-only**

In this problem, you will draw a series of diagrams that illustrate the execution of a program that operates on arrays. We will work on a similar problem in [Lab 2](#), and you may find it helpful to review our solutions to that problem (which will be posted after all of the labs have been held) before continuing.

Consider the following Java program:

```
public class ArrayTest {
    public static void foo(int[] a, int[] b) {

        // part 2: what do things look like when we get here?

        int[] c = {4, 3, 2, 1};
        for (int i = 0; i < c.length; i++) {
            a[i] = c[i];
        }
        b = c;

        // part 3: what do things look like when we get here?
    }

    public static void main(String[] args) {
        int[] a = {2, 4, 6, 8};
        int[] b = a;
        int[] c = new int[a.length];

        // part 1: what do things look like when we get here?

        foo(c, b);

        // part 4: what do things look like when we get here?

        System.out.println(a[2] + " " + b[2] + " " + c[2]);
    }
}
```

1. In section 4-1 of `ps2_partI` (see above), we have given you the beginnings of a

memory diagram. On the stack, we have included a stack frame for the main method. On the heap, we have included the array to which the variable `a` refers.

Complete the provided memory diagram so that it shows what things look like in memory **just before** the call to `foo()`.

To do so, you should:

- Click on the diagram and then click the *Edit* link that appears below the diagram.
 - Make whatever changes are needed to the diagram. Below the thick horizontal line, we have given you a set of extra components that you can use as needed by dragging them above the thick line and putting them in the correct position. *You may **not** need all of the provided components.*
 - You can also edit any of the values in an array by clicking on one of its cells and editing the text that is inside the cell.
 - Once you have made all of the necessary changes, click the *Save & Close* button.
2. In section 4-2 of `ps2_partI`, we have given you the beginnings of a memory diagram that you should complete to show what things look like in memory **at the start** of the execution of `foo()`—just before its first statement is executed.
 3. In section 4-3 of `ps2_partI`, we have given you the beginnings of a memory diagram that you should complete to show what things look like in memory **at the end** of the execution of `foo()`—just before it returns.
 4. In section 4-4 of `ps2_partI`, we have given you the beginnings of a memory diagram that you should complete to show what things look like in memory **after** the call to `foo()` has returned—just before the `print` statement executes in `main()`.

Note: We have included a method frame for `foo`, but it may or not be needed in this diagram; you should delete it if you determine that it would no longer be present in memory.

To check your answers: Java Tutor is an online tool that allows you to step through the execution of a Java program and visualize its execution. We've included a link to it on our [Resources page](#), or you can use this link: [Java Tutor](#)

Keep in mind that you could be asked to solve a similar problem on the exams, so you should only use Java Tutor after you have solved the problem on your own!

Problem 5: Two-dimensional arrays

10 points total; **individual only**

Overview

Two-dimensional arrays in Java are extremely similar to two-dimensional lists in Python.

In Python, a 2-D list is a list of lists:

```
grid = [ [2, 4],  
         [5, 7],  
         [8, 1] ]
```

In the same way, a 2-D array in Java is an array of arrays:

```
int[][] grid = { {2, 4},  
                 {5, 7},  
                 {8, 1} };
```

(Note: When we declare a variable for a 2-D array, the type of the elements is followed by two pairs of square brackets: `int[][]`)

Here is a comparison of other key operations for 2-D sequences:

operation	in Python	in Java
determining the number of rows	<code>len(grid)</code>	<code>grid.length</code>
determining the number of elements in row <i>r</i>	<code>len(grid[r])</code>	<code>grid[r].length</code>
determining the number of columns in a rectangular grid/matrix (in which all rows have the same length)	<code>len(grid[0])</code>	<code>grid[0].length</code>
accessing the element at row <i>r</i> , column <i>c</i>	<code>grid[r][c]</code>	<code>grid[r][c]</code>

Problems

Assume that you have a variable `twoD` that refers to a two-dimensional array of integers. Here is another example of such an array:

```
int[][] twoD = { {16, 12, 8, 4},  
                 {15, 11, 7, 3},  
                 {14, 10, 6, 2},  
                 {13, 9, 5, 1} };
```

1. (2 points) Write an assignment statement that replaces the value of 5 in the above array with a value of 20.
2. (3 points) Write a code fragment that prints the values in the rightmost column of the array to which `twoD` refers. Print each value on its own line. For the array above, the following values would be printed:

```
4
3
2
1
```

Make your code general enough to work on any two-dimensional array named `twoD` that has at least one value in each row.

3. (5 points) Write a code fragment that prints the values along a diagonal line from the lower-left corner to the upper-right corner. Print each value on its own line. For the array above, the following values would be printed:

```
13
10
7
4
```

Make your code general enough to work on any rectangular two-dimensional array in which the dimensions are equal – i.e., the number of rows is the same as the number of columns.

Submitting your work for Part I

Note: There are separate instructions at the end of Part II that you should use when submitting your work for that part of the assignment.

Submit your `ps2_partI.pdf` file using these steps:

1. If you still need to create a PDF file, open your `ps2_partI` file on Google Drive, choose *File->Download->PDF document*, and save the PDF file **in your ps2 folder**.
2. Login to Gradescope by clicking the link in the left-hand navigation bar, and click on the box for CS 112.
3. Click on the name of the assignment (PS 2: Part I) in the list of assignments on Gradescope. You should see a pop-up window labeled **Submit Assignment**. (If you don't see it, click the **Submit** or **Resubmit** button at the bottom of the page.)
4. Choose the **Submit PDF** option, and then click the *Select PDF* button and find the PDF file that you created. Then click the *Upload PDF* button.
5. You should see a question outline along with thumbnails of the pages from your uploaded PDF. For each question in the outline:
 - Click the title of the question.
 - Click the page(s) on which your work for that question can be found.

As you do so, click on the magnifying glass icon for each page and doublecheck that the pages that you see contain the work that you want us to grade.

6. Once you have assigned pages to all of the questions in the question outline, click the *Submit* button in the lower-right corner of the window. You should see a box saying that your submission was successful.

Important

- It is your responsibility to ensure that the correct version of every file is on Gradescope before the final deadline. ***We will not accept any file after the submission window for a given assignment has closed, so please check your submissions carefully using the steps outlined above.***
- If you are unable to access Gradescope and there is enough time to do so, wait an hour or two and then try again. If you are unable to submit and it is close to the deadline, email your homework ***before the deadline*** to cs112-staff@cs.bu.edu

Part II

50 points total

Important guidelines

The following guidelines apply to the programming problems (i.e., problems in which you submit a .java file) in this and all subsequent assignments.

- Your methods **must** have the exact names that we have specified, or we won't be able to test them. Note in particular that the case of the letters matters.
- If a method takes more than one input, you **must** keep the inputs in the order that we have specified.
- If you are asked to write a method that returns a value, make sure that it **returns** the specified value, rather than printing it.
- In your work on this and all subsequent problem sets, you may *not* use any of Java's built-in collection classes (e.g., `ArrayList`) unless a problem explicitly states that you may do so. We will be writing and using our own collection classes, and using the built-in classes will keep you from fully learning the key concepts of the course.
- More generally, you may not use *any* built-in class that we have not covered in lecture.
- Unless the problem states otherwise, you should limit yourself to `String` methods that we have covered in lecture: `length`, `equals`, `substring`, `charAt`, and `indexOf`.
- You may freely use code from the class website, assigned readings, and lecture notes (unless specifically directed otherwise), as long as you cite the source in your comments. However, you may **never** use code from anywhere else (e.g., elsewhere on the web, or code obtained from another person). You should also not share your own code with others. See our [collaboration policies](#) for more details.
- Each of your methods should be preceded by a comment that explains **what your method does** and **what its inputs are**. In addition, you should include a comment at the top of the file with your name, email, and a description of the program or class of objects represented by the class definition in that file.
- More generally, use good programming style. Use appropriate indentation, select descriptive variable names, insert blank lines between logical parts of your program, and add comments *as necessary* to explain what your code does. See the [coding standards](#) for more detail.
- At a minimum we expect that your submitted code will compile. If your code

does not compile during our tests, you will not receive any credit for that specific problem.

Problem 6: Practice with static methods, part I

16 points total; *pair-optional*

This is the only problem of the assignment that you may complete with a partner. See the [rules for working with a partner on pair-optional problems](#) for details about how this type of collaboration must be structured.

This problem asks you to write a series of static methods.

Getting started

1. If you haven't already done so, [create a folder](#) named ps2 for your work on this assignment.
2. Download the following file: [Methods6.java](#)

Make sure to put the file in your ps2 folder. If your browser doesn't allow you to specify where the file should be saved, try **right-clicking** on the link above and choosing *Save as...* or *Save link as...*, which should produce a dialog box that allows you to choose the correct folder for the file.

3. In VS Code, select the *File->Open Folder* or *File->Open* menu option, and use the resulting dialog box to find and open the folder that you created for this assignment. **(Note: You *must* open the folder; it is *not* sufficient to simply open the file.)**

The name of the folder should appear in the *Explorer* pane on the left-hand side of the VS Code window, along with the name of the `Methods6.java` file that you downloaded in step 2.

4. Click on the name `Methods6.java`, which will open an editor window for that file. Add the methods described below to the class in that file.

Make sure that you follow the important guidelines given above at the start of Part II. In addition:

- All of your method headers should begin with the keyword `public`.
- You do *not* need to handle bad inputs – inputs with a value that doesn't correspond to the description of the inputs provided in the problem.

Here are the methods you should implement:

0. a static method called `printVertical` that takes a `String` as its parameter and prints the characters of the string vertically — with one character per line. For example, a call of `printVertical("method")` should produce the following output:

```
m  
e  
t  
h  
o  
d
```

We have given you this example method in the starter file.

Testing your methods

To help you test your methods, we've provided a `main` method in `Methods6.java` that you can use to make appropriate test calls, and we've included a sample test call for `printVertical`.

You can see if the test calls produce the correct output by running the program. One way to do so in VS Code is to right-click on the name `Methods6.java` in the Explorer Pane and choose *Run*.

1. a static method called `printEveryOther` that takes a `String` as its parameter and prints every other character of the string – i.e., the first character, the third character, etc. For example, a call of `printEveryOther("method")` should produce the following output:

```
mto
```

This method should *not* return a value.

Hint: Use `printVertical` as a model.

To test your method, add one or more test calls to the `main` method, and run the program to see if you get the expected results.

2. a static method called `longerLen` that takes two `String` objects as parameters and *returns* the length of the longer of the two strings. If the two strings have the same length, you should return that length. For example:
 - a call of `longerLen("bye", "hello")` should return 5, because 5 is the length of "hello", which is the longer of the two strings.
 - a call of `longerLen("goodbye", "hi")` should return 7, because 7 is the length of "goodbye", which is the longer of the two strings.

This method should *not* do any printing; rather, it should determine the longer length and return it.

Hint: Because this method returns a value and doesn't print anything, you'll need to perform the necessary printing yourself when you test it. For example, you could add these lines to the `main` method:

```
int len = longerLen("bye", "hello");
System.out.println("the longer length is: " + len);
```

3. a static method called `secondIndex` that takes as its two parameters a string `s` and a character `c` and *returns* the index of the *second* occurrence of `c` in `s` (if any). If `s` does not have at least two occurrences of `c`, the method should return `-1`.

For example:

- a call of `secondIndex("banana", 'a')` should return `3`, because the second occurrence of `'a'` in `"banana"` occurs at position `3` in the string
- a call of `secondIndex("banana", 'n')` should return `4`, because the second occurrence of `'n'` in `"banana"` occurs at position `4` in the string.
- a call of `secondIndex("banana", 'b')` should return `-1`, because there is only one `'b'` in `"banana"`.
- a call of `secondIndex("banana", 'x')` should return `-1`, because there is no `'x'` in `"banana"`.

This method should *not* do any printing; rather, it should return the appropriate integer.

Additional testing

Once you have completed all of these methods, you should run a separate test program that we've created. This is an important step as it will help to identify any issues in your methods that might prevent us from grading them.

Here's how to use the test program:

1. Download the following file: [Test6.java](#)

Put it in your `ps2` folder with your `Methods6.java` file.

2. As needed, open your `ps2` folder using the *File->Open Folder* or *File->Open* menu option in VS Code. The name of the folder should appear in the *Explorer* pane on the left-hand side of the VS Code window, along with the names of both the `Methods6.java` and `Test6.java` files.
3. Right click on `Test6.java` and choose *Run* to compile and run the program. If the IDE reports errors in `Test6.java`, that probably means that at least one of your methods does not have the correct header — i.e., either the name, the return type, or the parameters are incorrect. Make whatever changes are needed to your methods to get `Test6.java` to compile and run.
4. Once `Test6.java` does compile and run, you should check the results that it produces; the correct results are given in the descriptions of the methods above.
5. Feel free to add additional test cases to the ones that we've provided.

Problem 7: Practice with static methods, part II

21 points total; **individual-only**

The guidelines at the start of Part II and at the beginning of Problem 6 also apply here.

Getting started

1. As needed, open your ps2 folder using the *File->Open Folder* or *File->Open* menu option in VS Code.
2. Select *File->New File*, which will open up an empty editor window.
3. Select *File->Save*, and give the new file the name `FunArray.java`.
4. In the new file, create a class called `FunArray`. In that class, implement each of the methods described below.

Here are the methods you should implement:

1. a static method called `firstX` that takes as its two parameters, an array of integers `vals` and an integer `x` and *returns* the index of the *first* occurrence of `x` in `vals` (if any). If `x` does not have an occurrence in `vals`, the method should return `-1`.

For example:

- a call of `FunArray.firstX(vals, 7)` where `vals = {1, 3, 5, 7}` should return `3`, because the first occurrence of `7` in `vals` occurs at index `3` in the array.
- a call of `FunArray.firstX(vals, 4)` where `vals = {1, 3, 5, 7}` should return `-1`, because there is no occurrence of `4` in `vals`.

2. a static method called `histogram` that takes in one integer array as its only parameter, which prints out a histogram-like asterisk representation of the array.

For example: a call of `FunArray.histogram(vals)` where `vals = {1, 3, 7, 5}` should print

```
*
***
*****
*****
```

In the case an element of `vals` is negative, no asterisks should be printed for that row of the histogram.

3. a static method called `reverse` that takes one integer array as a parameter and *returns* an array with the elements reversed.

For example:

- a call of `reverse(vals)` where `vals = {1, 3, 5, 7}` should return the array `{7, 5, 3, 1}`
4. a static method called `interleave` that takes two integer arrays `vals1` and `vals2` as parameters and *returns* the array formed by “interleaving” the integers in `vals1` and `vals2`. In other words, it should create a new array by alternating integers from the two strings: the first integer from `vals1`, followed by the first integer from `vals2`, followed by the second integer from `vals1`, followed by the second integer from `vals2`, etc.

For example:

- a call of `interleave(vals1, vals2)` where `vals1 = {1, 3, 5, 7}` and `vals2 = {2, 4, 6, 8}` should return the array `{1, 2, 3, 4, 5, 6, 7, 8}`

If one of the arrays is longer than the other, then the “extra elements” should go immediately after the interleaved elements.

For example:

- a call of `interleave(vals1, vals3)` where `vals1 = {1, 3, 5, 7}` and `vals3 = {-1, -2, -3, -4, -5, -6}` should return the array `{1, -1, 3, -2, 5, -3, 7, -4, -5, -6}`

Hints:

- Start with interleaving the two arrays of the same size. Then add to your solution to accomodate the extra elements.

Additional testing

Once you have completed all of these methods, you should use the following test program to test them: [FunArrayTest.java](#)

Follow a similar procedure to the one that you used with the `Test6.java` file for Problem 6.

Problem 8: A simple interactive program

18 points total; individual-only

Overview

This problem provides additional practice with conditional execution and static methods, and it also gives you a chance to write a simple program with repeated user interactions.

The program will allow the user to compute the shipping charge for a collection of one or items. The shipping charge for each item will be based on the following characteristics:

- the type of shipping (one-day, two-day, or standard)
- the type of the item being shipped (book, clothing, electronics, or toy)
- the weight of the item, rounded to the nearest pound

Rules

Here are the rules for determining the charge for a single item:

▪ one-day shipping

- toys: \$4.99, plus \$1.99 per pound
- electronics: \$5.99, plus \$1.99 per pound
- books and clothing:
 - if the item weighs 2 pounds or more: \$3.99, plus \$0.60 per pound
 - otherwise: \$4.99 (with no per-weight charge)

▪ two-day shipping

- toys: \$2.99, plus \$0.99 per pound
- electronics: \$3.99, plus \$0.89 per pound
- books and clothing:
 - if the item weighs 2 pounds or more: \$1.99, plus \$0.75 per pound
 - otherwise: \$2.99 (with no per-weight charge)

▪ standard shipping

- toys and electronics: \$1.99, plus \$0.80 per pound
- books and clothing:
 - if the item weighs 2 pounds or more: \$0.99, plus \$0.70 per pound
 - otherwise: \$1.99 (with no per-weight charge)

Example

Let's say that the user wants to use one-day shipping to ship three items: a 10-pound toy, a 2-pound book, and a piece of clothing that weighs less than 2 pounds. Using the rules for one-day shipping found above:

- The 10-pound toy would have a shipping cost of \$24.89 (\$4.99, plus \$1.99*10).
- The 2-pound book would have a shipping cost of \$5.19 (\$3.99, plus \$0.60*2).
- The piece of clothing would have a shipping cost of \$4.99 (with no per-weight charge because it weighs less than 2 pounds).

Thus, the total cost would be $\$24.89 + \$5.19 + \$4.99 = \35.07 .

(Note: For the sake of greater precision, you will actually perform most of the computations in cents – e.g., 499 instead of 4.99, and 99 instead of 0.99. Then, at the end of the program, you will convert the total cost to dollars. See below for more detail.)

Sample runs

Your final program should match the behavior of the sample runs available [here](#). Note that these sample runs are **not** comprehensive, and you should make sure to test other cases as well.

Getting started

1. Download the following file: [TerrierShipping.java](#)

Make sure to put it in your ps2 folder.

2. As needed, open your ps2 folder using the *File->Open Folder* or *File->Open* menu option in VS Code. The name of the folder should appear in the *Explorer* pane on the left-hand side of the VS Code window, along with the name of the `TerrierShipping.java` file that you downloaded in step 1.
3. Click on the name `TerrierShipping.java`, which will open an editor window for that file. You will see some starter code that we've provided. In particular, we've given you the beginnings of the `main` method, along with static methods for getting the type of shipping and the type of an item from the user.

Your tasks

0. Start by reading over the starter code that we've given you. Make sure that you understand it.
1. Complete the assignment statement in the `main` method for the variable `weight` so that it assigns an integer value obtained from the user. At the moment, the statement looks like this:

```
int weight = 0;
```

You should replace the `0` with the appropriate method call.

Important: You **must** use the `Scanner` object created at the start of the `main` method, and you may **not** create an additional `Scanner` object.

2. Add the code needed to compute the charge for a single item, based on the inputs provided by the user (which you may assume are all valid) and the rules found above. In particular, you must:
 - a. Add **at least three** additional static helper methods, each of which handles the computation of one type of shipping charge. These new methods should go between the `getItemType` and `main` methods that we have given you.

There is more than one possible way to divide up the work among these helper methods. For example, you could have:

- one method for each type of shipping (one-day, two-day, and standard)

or

- one method for each type of item (book, clothing, electronics, and toy).

See below for some important requirements that your methods must satisfy.

- b. Add code to `main` that uses conditional execution to call the appropriate helper method and assign its return value to the variable `itemCharge`.

Important requirements

Each of your static helper methods must:

- *take one or more parameters* that provide the necessary information about a single item
- compute and *return* the appropriate charge for that item ***in cents***. For example, instead of using a cost of 4.99 dollars, your method should use 499 cents. Keeping the charges in cents until the end of the program will prevent the loss of precision that can result when floating-point numbers (i.e., doubles) are added together.

The names of the methods are up to you, but you should pick names that describe what the methods are supposed to do.

3. Complete the line of code provided near the end of `main` so that it will convert the total charge from cents to dollars. At the moment, the statement looks like this:

```
double totalDollars = 0.0;
```

You should replace the `0.0` with the appropriate expression. Make sure that your computation is as precise as possible.

Submitting your work for Part II

Note: There are separate instructions at the end of Part I that you should use when submitting your work for that part of the assignment.

Submission checklist for Part II

- You have read the Java [coding conventions](#) and followed all guidelines regarding format, layout, spaces, blank lines, and comments.
- You have ensured that your methods have the names, parameters, and return types specified in the assignment.
- You have verified that your code satisfies all of the tests that we have provided, and you have conducted whatever other tests are needed to ensure that your code works correctly. If we are unable to compile your work for a given problem, you will not receive any credit for that problem.

You should submit only your `.java` file, please do not submit any `.class` files.

Make sure that you do **not** try to submit a `.class` file or a file with a `~` character at the end of its name.

Pair-optional problem

If you chose to work on this pair-optional problem with a partner, only one person from the pair should submit the file, and that person should add the other person as a group member following step 7 below.

Here are the steps:

1. Login to Gradescope as needed by clicking the link in the left-hand navigation bar, and then click on the box for CS 112.
2. Click on the name of the assignment (PS 2: Part II) in the list of assignments. You should see a pop-up window with a box labeled **DRAG & DROP**. (If you don't see it, click the **Submit** or **Resubmit** button at the bottom of the page.)
3. Add your file to the box labeled **DRAG & DROP**. You can either drag and drop the file from its folder into the box, or you can click on the box itself and browse for the files.
4. Click the **Upload** button.
5. You should see a box saying that your submission was successful. Click the (x) button to close that box.
6. The Autograder will perform some tests on your file. **Once it is done, check the results to ensure that the tests were passed.** If one or more of the tests did *not* pass, the name of that test will be in red, and there should be a message describing the failure. Based on those messages, make any necessary changes. Feel free to ask a staff member for help.

Note: You will **not** see a complete Autograder score when you submit. That is because additional tests for at least some of the problems will be run later, after the final deadline for the submission has passed. For such problems, it is important to realize that passing all of the initial tests does **not** necessarily mean that you will ultimately get full credit on the problem. You should always run your own tests to convince yourself that the logic of your solutions is correct.

7. If you worked with a partner and you are the one who is submitting the file:
 - Click on the **Add Group Member** link that appears below your name above the results of the Autograder.
 - In the pop-up box that appears, click on the **Add Member** link.
 - Type your partner's name or choose it from the drop-down menu.
 - Click the **Save** button.
 - Check to ensure that your partner's name now appears below your name above the results of the Autograder.
8. If needed, use the **Resubmit** button at the bottom of the page to resubmit your

work. **Important:** Every time that you make a submission, you should submit *all* of the files for that Gradescope assignment, even if some of them have not changed since your last submission.

9. Near the top of the page, click on the box labeled **Code**. Then click on the name of each file to view its contents. Check to make sure that the files contain the code that you want us to grade.

Important

- It is your responsibility to ensure that the correct version of every file is on Gradescope before the final deadline. ***We will not accept any file after the submission window for a given assignment has closed, so please check your submissions carefully using the steps outlined above.***
- If you are unable to access Gradescope and there is enough time to do so, wait an hour or two and then try again. If you are unable to submit and it is close to the deadline, email your homework ***before the deadline*** to cs112-staff@cs.bu.edu